

FACILITATOR COPY

Speaker Notes

All 72 slides, with timing, cues and contingencies

Workshop: Agentic AI for Teaching & Academic Processes

Symbiosis Institute of Technology, Pune

Facilitator: Mr. Dinesh Wadhwani

Day 2: 10 September 2026, 1.30 pm – 4.00 pm

Day 3: 11 September 2026, 9.30 am – 4.00 pm

Track A: Apify & Gemini · **Track B:** n8n (optional)

Companion: PPT1_Workshop_Slides.pptx

How to use this document

Speaker notes for all 72 slides, in order. The same notes are embedded in the .pptx under each slide — this printed version is for preparation and for holding in your hand.

The two-track structure

Track A is Apify. It is the spine. Nothing installs, everything runs in a browser, participants write the agent loop themselves.

Track B is n8n. It is the counterpoint, and it may not run — it depends on whether the lab machines were prepared in time. Your lab incharge confirms this on Tuesday evening.

DECIDE TUESDAY NIGHT, ANNOUNCE WEDNESDAY MORNING

If Track B is not available, say so once at the start of Day 2 without apology. It was designed to be droppable.

Then drop slides 12, 27's second half, 7B and 9B, and give the platform comparison verbally instead. You gain about two hours, which goes into the capstone.

Timing

Block	Time
Day 2 PM — opening and contract	10 min
Section 1 — What actually changed	12 min
Section 2 — Prompts	25 min
Lab Tests 1 & 2	65 min
Section 3 — Your first Actor	18 min
Lab Tests 3A & 3B	55 min
Day 3 AM — opening, keys, rate limits	20 min
Lab Tests 4A & 5A	70 min
Section 5 — From sequence to agent	22 min
Lab Test 6A	55 min
Section 6 — Tools	25 min
Lab Tests 7A & 7B	90 min
Section 7 — Multi-agent	25 min
Lab Test 8A	60 min
Section 8 — Apify cost and reach	12 min
Lab Tests 9A & 9B	60 min
Section 9 — Grounding	20 min
Lab Tests 10A & 11	120 min
Section 10 — Close	15 min

Presentation time is roughly **3 hours 15 minutes** across both sessions; the rest is lab work. **That ratio is deliberate.** If you run long on slides, cut slides, not lab time.

Eight non-negotiables

SAY THESE, REPEATEDLY

1. **Personal Gmail, not institutional.** Say it Day 2 opening, before Lab 1, and again Day 3. It will still catch two people.
2. **Add the API key, THEN rebuild.** Environment variables apply at build time. This catches a third of the room.
3. **Never put a key in an input field.** Input values are stored with the run.
4. **maxIterations = 5**, set before the first run.
5. **Three items, never thirty**, while building.
6. **Memory at 512 MB** before every Apify run.
7. **Export every dataset** — seven-day retention.
8. **Revoke both keys** at the end of Day 3. Walk them through it.

Track B only: **JavaScript Code node, never Python.** It fails on these machines with a missing-venv error.

If something breaks

Problem	Response
One or two people stuck	Assign a volunteer buddy, move on. Do not hold the room.
Whole room getting 429s	Confirm everyone used their own key. Pause 60 seconds, resume.
Nobody can reach the API	Proxy blocking <code>generativelanguage.googleapis.com</code> . Escalate; switch to pre-recorded captures.
<code>console.apify.com</code> blocked	Content filter on cloud IDEs. This is a Tier 1 blocker — escalate immediately.
Builds queuing badly	Stagger by row. Have half the room read the loop while the other half builds.
Running 20+ minutes behind	Drop 9A and 9B entirely. Protect 7A and 8A.
Running ahead	Extend the capstone. Never extend a lecture section.

The two moments to rehearse

The arithmetic failure, in the Tools section. Run it live, write the number on the whiteboard, count the rows with the room, then run it twice more and write those numbers beside the first. Three different answers to one arithmetic question. That whiteboard is the most persuasive object in the two days — leave it up.

The correct answer for the lab data is **55.00%, 11 of 20**. Most runs land in the 60–75 range, enough to move the attainment level by two bands.

Lab 8A — the moderator overruling the grader. The Actor logs a line reading ******* **MODERATOR OVERRULED THE GRADER: 8 -> 5 *****. Rehearse on your data until it reliably fires. If it does not, the payoff of the whole multi-agent section is lost.

One thing to check Monday

Pin the **exact free-tier Gemini model identifier** and write it on the board. The free-tier lineup changed during 2026 and every Actor in the manual takes the model as an input field. One wrong identifier across forty machines is forty 400 errors.

1. Agentic AI for Teaching

& Academic Processes

Open by acknowledging the morning. Dr. Nilima Zade has just spent 2.5 hours on Generative AI, agents and ethics. Do NOT re-teach any of it.

Say something close to: "This morning you saw what an agent IS. This afternoon you build one."

Set the contract immediately — by 4pm today every person in this room will have built something that works. Not seen a demo. Built one.

Watch the room: it is straight after lunch. Get them typing within 15 minutes or you lose them. Do not spend more than four slides before Lab Test 0.

2. The contract for these two sessions

State the deliverable plainly. Faculty have sat through AI sessions before that were all demo and no output; name that and distinguish this one.

The last bullet matters. Say it out loud: "I am going to show you this failing several times today. That is deliberate. If you leave here thinking this technology is reliable, I have done you harm."

Engineering faculty are sceptical by training. Leading with the failures buys credibility that a polished demo never will.

3. The order we build in

This slide justifies the entire structure. Spend a moment on it.

The failure mode of AI workshops is starting at maximum complexity. People nod, feel behind, and build nothing afterwards.

Say: "We are going to make you hit a wall before I hand you the tool that gets over it. That way the tool means something."

Tie it back — this is how they teach. You do not open a DBMS course with query optimisation.

4. What you leave with

Point out that these four are not illustrations — they are a curriculum. Each teaches a genuinely different concept.

Ask for a show of hands: who has an internal assessment to set in the next three weeks? Who has attainment to compute this semester? Usually most of the room. Name that: "Then you are not learning a technology today, you are doing work you already had."

If someone asks why not more use cases: seven shallow demos teach nothing. Four built properly transfer.

5. SECTION 1 — What actually changed

Short section, 10 minutes maximum. It is conceptual and they have had concept all morning.

Your job here is one idea: the difference between a workflow and an agent, and the fact that most of their work needs the former.

6. [statement] When you use a chatbot,

you are the runtime.

This is the framing line for the whole workshop. Deliver it slowly.

Ask: "How many of you have asked ChatGPT or Gemini to generate quiz questions?" Most hands. "How many of you still have that as something you can re-run in ten seconds today?" No hands.

That gap is what the next day and a half is about. Everything we build is an answer to one question: how much of that process moves out of your head and into something that runs on its own?

7. Three tiers — and confusing them wastes months

Walk down the table. Then stop on the last row and read it again.

"Cost to trust" is the row everyone skips and the row that decides whether any of this survives contact with real academic work. An agent is not free once built — it needs supervising forever.

If you only make one point in this section, make it this one.

8. [statement] Do the steps change

depending on what I find?

Give them the question and let it sit for a beat.

Then apply it live. Ask the room: "Computing CO attainment from a marks sheet — do the steps change?" No. Fixed formula. That is a workflow.

"A student asks you a doubt — do the steps change?" Yes, entirely. That is an agent.

They now have a decision rule they can use on Monday without any of the tools.

9. The advice nobody gives you

This is contrarian and they will remember it. Say it directly.

There is a whole industry incentive to make everything an agent. Resist it on their behalf.

Concrete example: someone will build an "attainment agent" that produces a slightly different number each run and cannot work out why. Because they built a creative system for a deterministic task.

Land the last line as the takeaway.

10. Two platforms, deliberately

Apify is the spine. Two reasons, and say both.

Nothing installs — it all runs in a browser tab, and their work sits in their own account rather than on a machine that gets reimaged.

And they write the loop themselves. n8n's Agent node, CrewAI, LangGraph all hide it. Configure one and you have learned an import statement. Write one and every framework afterwards is looking up method names.

Flag now that n8n is Track B and may not run. If the machines were not ready, those labs are dropped and nothing essential is lost. Say this without apology — it was designed that way on purpose.

11. [statement] n8n hides the loop.

The Actor shows it.

This is the framing for every comparison lab today and tomorrow.

They will build the first flow both ways in Lab 3, and the attainment agent both ways in Lab 7. Tell them now that the comparison is the exercise, not a bonus.

If Track B is not running, still say this line — it explains why you chose to have them write code rather than configure a node.

12. SECTION 2 — Prompts that survive contact

Do not let anyone treat this as preamble. Say explicitly: every agent we build tomorrow is a prompt with plumbing around it.

If the prompt is vague, adding tools and memory and multi-agent review produces vague output more elaborately and at greater expense.

This section is about 25 minutes then straight into Lab 1.

13. Where most faculty start

Actually run this on the projector. Do not describe it.

Read one or two of the generated questions out loud, then ask the room to tell you what is wrong with them. They will find it themselves — no CO tags, no marks, all at recall level, one topic they do not teach.

Letting them diagnose it is far more effective than you listing the faults.

14. [statement] Vague prompts do not produce

wrong answers. They produce average

This reframes what they just saw. The model did not fail — it was told almost nothing and filled every gap with the most typical answer from the whole internet.

Average is worse than wrong for teaching purposes, because wrong gets caught and average gets used.

15. RCTFG — five components

Give them the mnemonic and tell them where the faults usually are.

In practice the missing pieces are almost always F and G. People describe the task well and then leave the output shape and the refusal rules unstated.

Tell them: if a prompt is not working, one of these five is missing. Go through them in order and find which.

16. C — the line that does the most work

Emphasise the NOT TAUGHT line hard. This is the single most transferable thing in the section.

Models are trained on the whole internet's database material and drift toward the mean of it. Telling it what is OUT of scope is far more effective than hoping it stays in scope.

Tell them: every prompt you write for a course should carry this line. It takes thirty seconds and it stops out-of-syllabus questions appearing on your paper.

17. G — guardrails

In Lab 1 they will test this directly: ask for a 5NF question before adding guardrails (it complies) and after (it refuses).

Tell them that comparison is the most valuable thirty seconds in the lab. One paragraph of instructions is the difference between a tool that quietly puts out-of-syllabus questions on their paper and one that does not.

18. Three techniques worth the time

The justification trick is the one to press. It is the highest-value habit in the section and it comes back tomorrow in the attainment agent as a safety feature.

Say: "You are building yourself an audit trail. You will spot a bad tag in one second instead of never."

On Bloom's inflation — give the BCNF example out loud, it lands immediately with engineering faculty.

19. [statement] Iterate on the prompt,

not the output.

Simple, and consistently ignored. The instinct when output is wrong is to fix the output.

Make them promise to fix the prompt instead. This is the difference between using AI and building something with it.

20. What prompting cannot fix

Close the section here and deliver the top line slowly.

"Fluency and correctness are separate properties, and this technology delivers the first far more reliably than the second."

The relocation point is the honest sell. It is still a large saving. It is a different job. Pretending otherwise is how institutions get burned.

Then send them to Lab 1.

21. LAB TEST 1 & 2 — Prompt engineering, then make it permanent

Setup notes before releasing them:

They must be on a PERSONAL Gmail. Institutional accounts hide the Gems option entirely with no error message. Say this twice; it will still catch two people.

Gems are created web-only at gemini.google.com on a computer, not the mobile app.

Walk the room during Lab 1. The common stall is people writing a short prompt and expecting the layered improvement — push them to actually paste the full blocks.

Checkpoint before the break: everyone has generated a question paper for a subject they actually teach.

22. SECTION 3 — Your first Actor

Deliberately no AI in this section. Say why: they need to see data go in, get processed and come out before adding a component whose output varies between runs.

Debugging when you cannot tell whether the plumbing or the model is at fault is unpleasant. Get the plumbing solid first.

23. Every Actor is three sections

This is the whole platform model and it takes ninety seconds to teach.

Emphasise that the agent they build tomorrow has exactly this shape too. It is not a different kind of thing — the middle section is just longer.

Removes a lot of the intimidation of a new platform.

24. [statement] Put your prompts in the input schema.

The single most important design decision on this platform, and the thing that makes it bearable to work on.

Builds take one to two minutes. If prompts are hardcoded, prompt iteration — which is most of what they will do — runs at two minutes per attempt. Unusable.

If prompts are inputs, they can try six versions in two minutes.

Give them the question to ask of every value in their code: am I going to want to change this without changing the logic? If yes, it goes in the schema.

25. Two things that cost you money or time

Both are set on the Input tab, not in code, so there is no excuse for forgetting.

On retention: faculty have lost workshop output this way, intending to use it the following month. Say it as a real thing that happens, not a footnote.

26. [statement] Data in n8n is a list.

Every node runs once per item.

Only needed if Track B is running.

This is the idea that unlocks n8n and causes ninety per cent of beginner confusion. Draw it if you can.

Then make the contrast explicit — it is the same lesson as the agent loop, one level down.

27. LAB TEST 3A & 3B — Your first Actor — then the same thing in n8n

Insist on the hand verification. It is the habit that catches everything later.

Common stalls in 3A: JSON syntax error in input_schema.json breaking the build; forgetting to set memory before Start.

The step that matters is changing pass mark from 40 to 70 and re-running with no rebuild. Make sure everyone does it — that is where the fast-path lesson lands.

If Track B is not running, skip 3B and give the comparison verbally.

28. Agents that use tools,

check each other, and run on your own material

Recap in one line: "Yesterday's Gem answers. Today's agent acts."

The difference is three things — it can call a TOOL, it can LOOP, and a second agent can CHECK it. Everything today is one of those three.

Then go straight to the setup gate. Nobody moves to Lab 4 until every machine has printed a response from the API. Two people will still be stuck — assign a buddy and move on. Do not hold 38 people for 2.

29. What changes when you add an AI call

The second row is the whole slide. Read it aloud.

"A broken deterministic Actor throws an error and stops. A broken AI Actor completes successfully and hands you something that looks correct."

Every design decision for the rest of today follows from that sentence. Come back to it when you introduce the moderator this afternoon.

30. Two things about the API key

The rebuild gotcha will catch about a third of the room. Say it twice and it will still catch some.

On the input-field point: this is a real security habit, not pedantry. Exported code and run records travel.

Remind them again at the end of Day 3 to revoke both keys.

31. Rate limits — this will affect you today

Spend a full two minutes here. It prevents the most common failure of the day.

Nothing today runs calls in parallel. Everything is sequential with a delay. Slow and complete beats fast and half-failed.

If someone hits the daily quota, they are done for the day on that key — pair them with a neighbour rather than letting them keep clicking.

32. The parser that prevents the most failed runs

Models wrap JSON in code fences habitually despite being told not to. Fighting it is a waste of time; handle it.

Three lines of defence, and tell them to put it in every parser they ever write.

Also flag the try/catch around each call: without it, one malformed response kills the whole run and they lose everyone else's results.

33. LAB TEST 4A & 5A — An Actor that calls Gemini, then structured output

The prompt-iteration step is the one to push. Make them do four iterations on the Input tab and watch the clock — two minutes for four attempts. That is only possible because the prompt is an input.

In Lab 5A there is a deliberate trap: Bhavesh scores 42 per cent, the rule says "strictly below 40". Some models still set `needs_meeting` to true — applying judgement over the stated rule. Make sure people notice it. That is the silent-error class the whole afternoon is about.

Also have them deliberately break the parser and watch the run survive.

34. SECTION 5 — From flow to agent

Open with the wall. Show them a question a flow cannot answer, then hand them the thing that can.

35. [statement] "I did not understand the 3NF example

from Tuesday's class.

Run this against a plain flow first so they see the generic answer.

Then decompose why it fails: steps 1 and 5 are the problem. In a flow you decide in advance what happens at every step. Here, what happens depends entirely on what the student said and how they respond.

This is the boundary. Everything after this slide is about crossing it.

36. Three additions turn a flow into an agent

Emphasise the parenthetical on Tools: you do not tell it which to use. That delegation of choice is the definition.

On the loop — this is what costs money and time. Three to ten API calls instead of one. Flag that now, because the next slide is about the bill.

37. [statement] An agent is a model choosing

its own path, one step at a time

The definition to leave on screen while they build.

If anyone asks about ReAct, planner-executor or the agent taxonomies from yesterday morning — those are names for variations on this loop, and they will see the loop itself in code in ten minutes. Do not go down the taxonomy route.

38. The entire loop, in five steps

This slide is the payoff of choosing to write the loop rather than configure a node.

Say plainly: "Everything in agent engineering is refinement on those five lines of structure."

Then tell them what they get from having written it — they can change the stopping condition, add a validation step, log whatever they want. None of that is possible inside a node you cannot open.

39. A tool is a description and a function

Three rules for a tool description: what it does, exactly what it needs, and when to use it.

"Does calculations" is useless. The one on screen is usable.

Point out that the ONLY thing that changes between the agents they build today is this tools object. The sixty lines of loop stay identical. That is the lesson.

40. Set maxIterations to 5. Before your first run.

Make them set it in the lab as step one, not step nine.

The second instruction is less obvious and just as important — over-cautious looping is more common than runaway looping in practice.

In Lab 6A they deliberately set it to 1 and watch the cap fire. Make sure they do that.

41. [statement] Give the agent fewer tools

than you think it needs.

Counterintuitive and true. The instinct is to give it every capability available.

"Every tool you add makes every decision harder."

Add a third only when they have watched it genuinely fail for want of it.

42. LAB TEST 6A — Your first agent loop — written by hand

Bullet one is not optional. Give them four silent minutes to read the loop and find the five parts before anyone clicks Build.

Then press the trace point hard. It is the only way to understand WHY an agent did something. An agent that gave a wrong answer usually shows exactly where it went astray.

Debugging by re-reading output is guesswork. Debugging by reading the trace is diagnosis.

The negative test in bullet four is the one people skip. If the tool fires when it should not, their description is too broad — and they fix it on the Input tab, no rebuild.

43. SECTION 6 — Tools — the thing that matters most

The most important section of the workshop. Do the failure demo properly and do not rush it.

If the room only remembers one thing from two days, it should be this section.

44. Watch this before you read ahead

Do this live. Actually run it on the projector, write the number on the whiteboard, then have the room count with you.

Then run it twice more and write those numbers next to the first. Three different answers to the same arithmetic question.

That whiteboard is the most persuasive object in the two days. Leave it up for the rest of the session.

The correct answer for the lab data is 55.00 per cent, 11 of 20. Most runs give something in the 60 to 75 range — enough to move the attainment level by one or two bands.

45. Why this matters more in academic work

This is where the room goes quiet. Let it.

Engineering faculty already suspect LLMs are bad at arithmetic — you are confirming a suspicion and then handing them the fix, which is a much better position than surprising them.

The eight-months line is the one that lands. Say it slowly.

46. [statement] The model decides.

The tool computes.

The principle, generalised. Anywhere a number, a date, a lookup or a fact must be exactly right, it comes from a tool.

This single principle separates an agent you can put in front of a department from a demonstration.

Note what did NOT change: same model, same data. Only the division of labour changed.

47. The instruction that does the work

Tell them: an agent willing to do EASY arithmetic itself will also be willing to do hard arithmetic itself, and the hard ones are where it gets things wrong.

So the prohibition has to be absolute, including simple percentages. No carve-outs.

In Lab 7 they will test whether it obeys. If it computes a percentage itself, the instruction needs strengthening.

48. [statement] Make the agent state its inputs

before its result.

The failure a tool does NOT prevent. Make this explicit.

A tool guarantees the calculation is correct. It does not guarantee the agent passed the right arguments. It could correctly call `calculate_attainment` while passing q2 and q5. The maths would be flawless and the answer wrong.

Same trick as the CO justifications yesterday. Point that back — it is a pattern, not a one-off.

49. LAB TEST 7A & 7B — The attainment agent — built twice

The most important lab. Enforce Part A — people will want to skip to the working version.

The CO7 test in the last bullet is the one that matters most. If it invents a number for a non-existent CO, that is a fabricated figure in an accreditation document. Make everyone run that test and confirm the refusal.

Float continuously during this lab. The common technical stall is the tool input parsing — the query variable arrives as a string and needs `JSON.parse`.

50. SECTION 7 — Multi-agent

Second-longest section. The payoff moment is the moderator overruling the grader — rehearse until that reliably happens on your demo data.

51. Two failures a single grading agent always commits

This is the ethical core of the day, and it is placed here deliberately — attached to a live consequential decision rather than floating on a slide at 9 am.

The sample data is built for this: S003 is correct but awkward, S004 is fluent and wrong. A single grader typically scores S004 at or above S003.

Have them find it themselves in the lab rather than telling them the answer.

52. The critic's instructions — note the last line

Criteria 3 and 4 exist specifically to catch the bias from the previous slide. Point that out — it is a designed countermeasure, not a generic checklist.

The final paragraph is the load-bearing one. If their moderator rubber-stamps in the lab, this is what they add.

53. What multi-agent costs you

Make them do this arithmetic against their actual class size before deploying anything.

A flow that works on five and dies on eighty fails at exactly the moment they needed it.

Then the honest counterpoint: most of the time multi-agent is not worth it. If a task has one right answer and a deterministic check exists, use the check. Verifying a paper totals 20 marks needs a line of JavaScript, not a reviewing agent.

Use a second agent only for judgements that genuinely require judgement.

54. Before this touches a real mark

Deliver this as practical advice, not a legal disclaimer. It is the difference between a pilot that survives and one that gets shut down after a complaint.

The two quoted positions at the end are the takeaway. Suggest they write the first one into any proposal they put to their department.

If someone says their institution has no policy — that means they are setting precedent. Set it deliberately and in writing.

55. LAB TEST 8 — Multi-agent rubric grading

The comparison in bullet five is the highest-value exercise available to them. It teaches exactly how far to trust this, on material with a ground truth they produced.

If someone's moderator agrees with everything, give them the strengthening line from the manual: "You must find at least one point of disagreement."

Quota warning: three agents across five submissions is about 15 calls per run. One full run is about a minute of quota. Tell them not to run it repeatedly out of curiosity.

56. SECTION 8 — Apify — reach and cost

Short section, 15 minutes. The cost trap is the whole point — everything else is mechanics they can read in the manual.

57. The Apify pricing trap

Show the arithmetic on screen. It is the most concrete thing in the section.

Also flag the two things that save them money and work:

Memory setting — Apify bills 1 CU per GB-hour. At 512 MB a run costs a fraction of a cent; at 4 GB it costs eight times as much for identical work. Always set the lowest memory that works.

Seven-day retention on free tier. Anything they build this week vanishes by mid-next-week unless exported. Faculty have lost workshop output this way.

58. Two ways to connect Apify — one is usually right

The instinct is to make everything a tool. Resist it on their behalf.

If they always need the data, fetching it conditionally adds a failure mode for no benefit.

Lab 10 builds the tool version because it is the one worth practising. In their own work they should ask which pattern fits.

59. LAB TEST 9A & 9B — A web-fetch tool, then bridge the two platforms

Registration link is in the lab manual and on the board. Personal email, no credit card.

The failure-honesty test at the end of Lab 10 matters: give it a URL that does not exist. If it answers about the topic anyway from training data and presents it as if it came from the page, the instructions need strengthening. They would have no way to tell — the citation would be there and the content would not be from the source.

60. SECTION 9 — Grounding and guardrails

Flag the stakes at the top. This is the one build where getting it wrong affects someone other than the faculty member.

61. Two ways to ground an agent

Save them three days. Reaching for the vector pipeline first is the most common way to spend a week building something a five-minute approach handles.

Only go further when grounding across a whole programme rather than one course.

62. The three instructions that make grounding work

Explain three properly. Their notation, their simplifying assumptions, their conventions may differ from textbook consensus.

The agent must follow THEM, and flag the difference rather than silently correcting them in front of a student.

Concrete example from the sample material: the course uses 3NF as the practical target and a specific FD notation. A generic model would drift to the textbook version.

63. The Socratic constraint is part of the build

State plainly why this is not a refinement to add later: colleagues will reject the tool outright, and correctly, if it hands out solutions.

The lab tests it under pressure — "I don't have time, just give me the full answer." A constraint that collapses under mild pressure is not a constraint. Make sure everyone runs that test.

Also flag the escalation path: grade queries, deadlines, and any mention of personal difficulty go to a human immediately. Design that FIRST, not last.

64. [statement] Log every question students ask.

Faculty who deploy these consistently report this as the most valuable output.

The answers help students. The QUESTIONS tell them how to teach the course better next semester.

This is a genuinely good closing note for the section — it reframes the agent as a teaching instrument rather than a labour-saving device.

65. LAB TEST 10A & 11 — Doubt-solving agent, then your capstone

On the distress test: an agent that tries to counsel a student on its own is a serious problem, not a feature. Correct behaviour is a warm, prompt handoff. Verify it.

For the capstone: starter workflows for all four lanes are in the shared folder. Nobody starts from a blank canvas.

Recruit 2 to 3 volunteer helpers over lunch — identify strong participants during Lab 8.

Hard stop at 60 minutes is not negotiable. Every year somebody demos an untested workflow and it fails live.

66. SECTION 10 — From workshop to practice

Fifteen minutes. Not a futurism slide — a commitment device.

67. Why most of this will not survive

Be blunt. They have all been to workshops that produced nothing, and naming it is more useful than pretending this one is different by default.

Then give them the specific remedy on the next slide.

68. This week — pick one

Row three is the highest-value exercise available to them and worth calling out.

Grade five themselves, run the agent, compare. They learn exactly where it agrees, where it diverges, and how far to trust it — on their own material with a ground truth they produced.

Weeks 1-2: one workflow, one course, used in earnest, note every failure. Weeks 3-4: fix what failed, show one colleague. Their questions find the assumptions you cannot see.

69. If this is to go beyond you

The last bullet is a concrete, actionable thing for whoever runs the department. Mention it specifically to the HoD if present.

On ownership — resist the committee instinct. Name one person or it dies.

Cost honesty: free tier is genuinely sufficient for one teacher on their own courses. Student-facing at cohort scale needs a paid tier. Say so rather than overselling.

70. [statement] The durable skill is not any tool

in this workshop.

Close on this rather than on a forecast. Forecasts age badly and they will remember this line.

Then the final ask on the next slide.

71. One thing, by Friday

Actually make them write it. Pause, hand out paper if needed, wait thirty seconds in silence. The pause is the point.

Then the housekeeping — the key revocation is not optional on shared machines. Walk them through it if there is time.

Propose a follow-up in four to six weeks where people bring what actually worked and what quietly died. The second category is more useful than the first.

Thank the lab team by name. They did two days of unglamorous work to make this possible.

72. Now go and build

something small.

Leave the last slide up during questions.

Point them at the shared folder one more time — study material, lab manual, starter workflows, sample data.

Remind them one final time about key revocation before they walk out.